

Guillaume ?
se demande 🤔





le TDD ?

LE TDD EST-IL MORT AVEC L'AGENTIC CODING ?

UNE PERSPECTIVE ÉVOLUTIVE SUR LE DÉVELOPPEMENT LOGICIEL

LE TDD CLASSIQUE (HUMAIN)

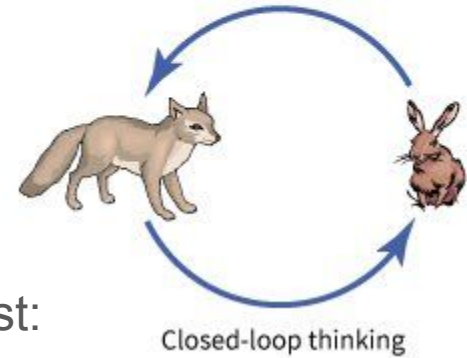
1.  **Rédiger le Test (Rouge)**
 - Intention humaine explicite
2.  **Implémenter le Code (Vert)**
 - Cycle court et rapide
3.  **Refactoriser le Code**
 - Intention humaine explicite
 - Cycle court et rapide
 - Confiance dans la base de code

AGENTIC CODING LOOP (IA)

1.  **Agent Reçoit le Prompt / Contrat**
 - Automatisation de l'intention humaine
2.  **Génère et Exécute les Tests**
 - Génération massive de tests unitaires/intégration
3.  **Itère Automatiquement l'Implémentation**
 - Automatisation de l'intention humaine
 - Génération massive de tests unitaires/intégration
 - Itération rapide à grande échelle

on s'arrête là ?

le TDD est une boucle de réflexion



Avant même d'avoir un test rouge, la question est:

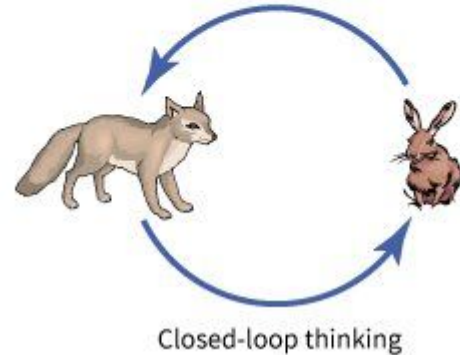
Comment je pose le code du test ?

- Quel(s) objet(s) j'expose ? => ARRANGE
- Quelle est l'état de départ => ARRANGE
- Quelle situation je déclenche ? => ACT
- Quel comportement j'attends ? => ASSERT

mais ça ne s'arrête pas là !

Réfléchir avant de coder

Pour que le test soit **ROUGE** pour les bonnes raisons :

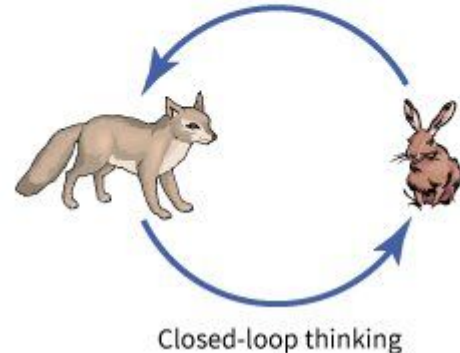


ARRANGE

- est ce que j'ai exposé le bon objet (ou la bonne surface de code) ?
 - détection des dépendances exotiques ou toxiques
 - besoin de mieux isoler votre code et ses responsabilités (social testing)
- est-ce que l'état de départ est simple et clair ?
 - trop de setup = dépendances aux données dans la logique
 - = logique mal répartie
 -  attention aux valeurs constantes ou en configuration

Réfléchir avant de coder

Est-ce que le test est **ROUGE** pour les bonnes raisons ?



- Comment se passe mon action (**ACT**)
 - Est ce que des exceptions sont levées alors que je ne les attendais pas
 - feedback immédiat 🍀
 - Est ce que je dois enchaîner plus d'une action pour avoir un résultat?
 - si oui = **couplage temporel** 🚫
 - Est ce que mon action me renvoie un résultat immédiat ?
 - ⚠️ tester "void" c'est impossible 🚫 il faut un minimum de valeur de retour
- Est ce que mes assertions sont les bonnes (**ASSERT**) ?
 - est ce que ça a du sens métier ou correspond au comportement attendu ?
 - est ce que je ne suis pas en train de vérifier un effet connexe ou décalé
 - est ce que je ne suis pas en train de faire un test d'intégration ?
 - je teste A mais je dois vérifier B ? 🚫 un **couplage** est mis en évidence

Bénéfices

en vrac:



le code est prévu pour les tests

(plus la peine de galérer pour savoir ce qui est public/privé)



les tests étant unitaire, cela force votre code à l'être également (Single Responsibility Principle, Unix Philosophy)



le test est là pour toujours 🎰 non régression



le test explique pourquoi le code est là, comment il a été pensé, qu'est ce qu'on attends de lui 🎰 documentation vivante



SYNTHÈSE



TDD fournit la boucle d'interaction
qui relie humain et code



TDD & AGENTIC CODING : GARANTIR LA QUALITÉ ET LA RÉSILIENCE

VÉRIFIER L'ÉMERGENCE DU DESIGN CONTRE LE MUR DES EXIGENCES

LE RÔLE CLASSIQUE DU TDD (HUMAIN ET GUIDÉ)

1.  **VÉRIFICATION INTENTIONNELLE**
 - Valider le design émergeant
2.  **REFACTORING GUIDÉ**
 - Assurer la maintenabilité et la clarté
3.  **TEST DE RÉSILIENCE**
 - Confirmer l'adéquation aux exigences futures

LE TDD AVEC AGENTIC CODING LOOP (SYNERGIE ÉVOLUTIVE)

1.  **GÉNÉRATION MASSIFIÉE INTELLIGENTE**
 - Automatiser les tests de régression
2.  **DÉTECTION DE LA "DETTE IA"**
 - Identifier le code non justifié par un test
3.  **VALIDATION DE LA STRUCTURE**
 - Vérifier si le design est contraint ou émerge

**Agentic TDD : un élément indispensable du harnais (harness)
sur les parties critiques de votre produit logiciel.**